ASSIGNMENT 2 – TESTING REPORT

Student Course Registration System

Name: Faiza Qadeer
Class: BS-SE 5B
Course: Software Quality Engineering
Project: Student Course Registration System

1. INTRODUCTION

The purpose of this testing report is to evaluate the correctness and quality of the Student Course Registration System.

The system was tested using black-box and white-box testing techniques. Different test cases were designed and executed to identify errors, verify functional requirements, and examine the internal structure of the program.

The assignment also involved GitHub for version control, GitHub Issues for defect tracking, Pull Requests for reviewing changes, and GitHub Actions for automated testing.

2. TESTING OBJECTIVES

The main objectives of this assignment were:

1. Verify that the system meets its functional requirements.
2. Identify defects in course registration and withdrawal operations.
3. Apply equivalence partitioning and boundary value analysis.
4. Use decision table testing to test different conditions.
5. Analyze program logic using Control Flow Graphs (CFGs).
6. Calculate cyclomatic complexity.
7. Perform white-box testing and MC/DC analysis.
8. Measure code coverage using pytest-cov.
9. Use GitHub for version control and collaboration.
10. Automate test execution using GitHub Actions.

3. BLACK-BOX TESTING

3.1 Introduction

Black-box testing was performed to check the functionality of the Student Course Registration System without examining its internal code structure.

The test cases were designed according to the functional requirements of the system.

3.2 Testing Techniques Used

The following black-box testing techniques were applied:

1. Equivalence Partitioning

Input values were divided into valid and invalid groups. Test cases were selected from these groups to check the system's behavior.

2. Boundary Value Analysis

Boundary values were tested because errors commonly occur at the limits of input ranges.

Examples included:

- Credit limit of 18.
- Current credits from 0 to 18.
- Course credit hours from 1 to 4.
- Outstanding dues greater than zero.
- Course enrollment equal to its capacity.

3. Decision Table Testing

Different combinations of conditions were used to check registration decisions, including outstanding dues, prerequisites, credit limits, course availability, and existing registration.

3.3 Black-Box Test Execution Results

A total of 44 black-box test cases were executed using pytest.

Result	Number
Total test cases	44
Passed	35
Failed	9

The nine failed tests revealed problems in the original implementation.

3.4 Defects Identified

Five main defect areas were identified:

1. Credit Limit Boundary Defect

The system did not allow a student to register for a course when the total credit hours became exactly 18. However, the requirement allows a maximum of 18 credit hours.

2. Prerequisite Validation Defect

The system checked whether at least one prerequisite was completed instead of checking whether all required prerequisites were completed.

3. Outstanding Dues Defect

The system blocked registration only when outstanding dues were greater than 1000. According to the requirement, any positive outstanding dues should block registration.

4. Course Capacity and Waitlist Defect

The system did not correctly identify a full course. When enrolled students equal course capacity, the student should be placed on the waitlist.

5. Withdrawal Enrollment Count Defect

When a student withdrew from a course, the system removed the course from the student's registered courses but did not decrease the course enrollment count.

3.5 Black-Box Testing Conclusion

Black-box testing helped identify functional problems in the registration system. Equivalence partitioning, boundary value analysis, and decision table testing helped create test cases for different input conditions and system behaviors.

4. WHITE-BOX TESTING

4.1 Introduction

White-box testing was performed to examine the internal logic and structure of the program.

The source code was analyzed to understand decision conditions, execution paths, and possible errors.

A separate white-box test file was created:

`tests/test_whitebox.py`

4.2 Functions Tested

The following functions were selected for white-box testing:

1. `check_credit_limit()`
2. `prerequisites_satisfied()`
3. `registration_decision()`
4. `register_course()`
5. `withdraw_course()`

4.3 Control Flow Graphs

Control Flow Graphs (CFGs) were created to represent the execution flow of the selected functions.

Each graph contains nodes representing statements or decisions and edges representing the possible flow of execution.

CFGs helped identify:

- Sequential statements.
- Conditional statements.
- True and false branches.
- Independent execution paths.
- Unreachable paths.

The CFG diagrams are included in the assignment documentation.

4.4 Cyclomatic Complexity

Cyclomatic complexity measures the number of independent paths in a program.

The formula used was:

V(G) = E - N + 2P

Where:

- E = Number of edges.
- N = Number of nodes.
- P = Number of connected components.

The calculated complexity values were:

Function	Cyclomatic Complexity
check_credit_limit()	3
prerequisites_satisfied()	2
registration_decision()	8
register_course()	4
withdraw_course()	2
Total	**19**

The total of 19 represents the sum of the individual function complexities, not the complexity of one combined control flow graph.

4.5 Independent Paths and MC/DC Testing

Independent execution paths were identified using the Control Flow Graphs.

MC/DC (Modified Condition/Decision Coverage) was also considered to check whether individual conditions could independently affect the overall decision.

The registration decision was analyzed using conditions such as:

- Outstanding dues.
- Already registered status.
- Prerequisite completion.
- Credit limit.
- Course availability.

The WAITLISTED decision is unreachable in the original implementation because the capacity validation and availability condition do not correctly support the required full-course behavior.

Therefore, complete MC/DC coverage of that decision could not be demonstrated against the original implementation.

4.6 White-Box Test Execution Results

A separate test file was created for white-box testing.

The results were:

Result	Number
Total white-box tests	14
Passed	14
Failed	0

All 14 white-box test cases passed successfully.

4.7 Latest Combined Test and Coverage Results

After adding the white-box tests, the complete test suite contained 58 test cases.

Result	Number
Total test cases	58
Passed	49
Failed	9

The latest GitHub Actions coverage report showed:

Coverage Metric	Result
Total statements	57
Missed statements	3
Total branches	26
Partially covered branches	2
Overall coverage	92%

The nine failed tests are related to the five previously identified defect areas in the original implementation.

All 14 white-box tests passed. The original implementation was kept unchanged so that the identified defects could be documented and tested.

4.8 White-Box Testing Conclusion

White-box testing helped analyze the internal structure and logic of the program.

Control Flow Graphs, cyclomatic complexity, independent paths, MC/DC scenarios, and code coverage were used to evaluate the implementation.

5. GITHUB AND VERSION CONTROL

5.1 GitHub Repository

A public GitHub repository was created for the assignment.

Repository: SQE-Assignment-2

Repository Link:

https://github.com/faiza-qadeer/SQE-Assignment-2

The repository contains the source code, test cases, requirements, testing documentation, and submission files.

5.2 Branches

The following branches were used:

1. `main`
2. `testing-improvements`
3. `submission-format`

The main branch contains the final merged project. The additional branches were used to organize documentation, workflow, testing, and submission-format changes.

5.3 Git Commits

Git commits were used to record project changes and maintain version history.

Some important commits included:

Commit	Description
`87dd38a`	Add Part A and Part B testing
`10d1956`	Document defects tracked in GitHub Issues
`e126b10`	Add GitHub Actions automated testing
`b6d4a64`	Add final assignment report
`ae78b6e`	Add white-box tests and assignment documentation files
`571c187`	Update final report with combined test results

5.4 GitHub Issues

Five GitHub Issues were created to document the identified defects.

1. Issue #1 – Fix credit limit boundary condition.
2. Issue #2 – Fix prerequisite validation logic.
3. Issue #3 – Fix outstanding dues validation.
4. Issue #4 – Fix course capacity and waitlist logic.
5. Issue #5 – Fix enrollment count during withdrawal.

These issues provided a record of the defects and their expected corrections.

5.5 Pull Requests

Four Pull Requests were created and merged into the main branch.

Pull Request	Description
PR #6	Document testing defects and improvements
PR #7	Add GitHub Actions automated testing
PR #8	Add final assignment report
PR #9	Add required submission files and white-box tests

Pull Requests were used to review and merge changes from the working branches into the main branch.

6. GITHUB ACTIONS – AUTOMATED TESTING

6.1 Introduction

GitHub Actions was configured to automate the execution of test cases.

The workflow file is located at:

`.github/workflows/tests.yml`

6.2 Workflow Operations

The workflow performs the following operations:

1. Checks out the repository.
2. Sets up Python 3.12.
3. Installs the required dependencies.
4. Runs pytest.
5. Measures code coverage using pytest-cov.

The workflow is triggered by pushes to the `main` and `testing-improvements` branches and pull requests targeting `main`.

6.3 GitHub Actions Results

The latest workflow run successfully executed all 58 tests but reported failure because nine tests failed due to known defects in the original implementation.

The workflow itself was successfully triggered and executed.

The results were:

- 49 tests passed.
- 9 tests failed.
- Overall coverage was 92%.

The failures were consistent with the previously documented defects.

7. PROJECT FILE STRUCTURE

The GitHub repository contains the following important files:

Main Project Files

- `registration.py`
- `README.md`
- `REQUIREMENTS.md`
- `requirements.txt`
- `Student_Registration_SRS.docx`

Testing Files

- `tests/test_blackbox.py`
- `tests/test_whitebox.py`

Documentation Files

- `docs/blackbox_results.md`
- `docs/blackbox_test_design.md`
- `docs/blackbox_test_design.xlsx`
- `docs/decision_table.xlsx`
- `docs/cfg.png`
- `docs/whitebox_testing.md`
- `docs/final_report.md`
- `docs/testing_report.pdf`

Automation

- `.github/workflows/tests.yml`

8. OVERALL CONCLUSION

The Student Course Registration System was evaluated using black-box and white-box testing techniques.

Black-box testing helped identify functional defects, while white-box testing examined the internal program structure, execution paths, and code coverage.

A total of 58 test cases were executed. Out of these, 49 passed and 9 failed because of known defects in the original implementation. All 14 white-box test cases passed.

Five main defect areas were identified and documented through GitHub Issues. GitHub branches, commits, Pull Requests, and GitHub Actions were also used to manage and automate the project.

Overall, this assignment demonstrated how different Software Quality Engineering techniques can be combined to evaluate software quality, identify defects, measure coverage, and document areas that require improvement.